
✅ 1) SQL-Level Deduplication (Window Functions)

Most common pattern:

```
SELECT *
FROM (
  SELECT *,
         ROW_NUMBER() OVER (
           PARTITION BY order_id
           ORDER BY updated_at DESC
         ) AS rn
  FROM dataset.orders_staging
)
WHERE rn = 1;
```

Why it works

- Groups by business key (order_id)
- Keeps latest record using timestamp

Use when

- Batch loads
- Periodic cleanup
- Staging → curated layer

✅ 2) MERGE-Based Idempotent Loads (Upserts)

Prevents duplicates while loading:

```
MERGE dataset.orders T
USING dataset.orders_staging S
ON T.order_id = S.order_id
WHEN MATCHED THEN
  UPDATE SET
```

```
    T.status = S.status,
    T.updated_at = S.updated_at
WHEN NOT MATCHED THEN
  INSERT (order_id, status, updated_at)
  VALUES (S.order_id, S.status, S.updated_at);
```

Benefits

- Handles inserts + updates
- Safe for reprocessing same data
- Production-grade approach

✅ 3) Streaming Deduplication Using InsertId

For streaming inserts:

insertId = f"{order_id}-{event_time}"

BigQuery ignores rows with same insertId within a time window.

Use when

- Pub/Sub → Dataflow → BigQuery streaming

✅ 4) Partition-Level Dedup (Cost Efficient)

When data arrives daily:

```
CREATE OR REPLACE TABLE dataset.orders PARTITION BY DATE(updated_at)
AS
SELECT *
FROM (
  SELECT *,
        ROW_NUMBER() OVER (
          PARTITION BY order_id, DATE(updated_at)
          ORDER BY updated_at DESC
        ) rn
```

```
  FROM dataset.orders_raw
)
WHERE rn = 1;
```

Reduces scan cost by working per-partition.

✅ 5) Hash-Based Dedup (Whole Row)

```
SELECT *
FROM (
  SELECT *,
         ROW_NUMBER() OVER (
           PARTITION BY FARM_FINGERPRINT(TO_JSON_STRING(t))
           ORDER BY load_time DESC
         ) rn
  FROM dataset.raw t
)
WHERE rn = 1;
```

Useful when no natural key exists.

✅ 6) Enforce Uniqueness via Scheduled Job

BigQuery doesn't support unique constraints.
So we simulate using:

- Scheduled MERGE
- Scheduled CREATE OR REPLACE with ROW_NUMBER
- Dataform / dbt tests

Example dbt test:

```
SELECT order_id, COUNT(*)
FROM {{ ref('orders') }}
GROUP BY order_id
HAVING COUNT(*) > 1
```

🎯 Best Practice Architecture

Raw Layer (duplicates allowed)

↓

Staging (ROW_NUMBER dedup)

↓

Curated (MERGE idempotent)

⚡ Interview One-Liner

I ensure deduplication in BigQuery using window functions with ROW_NUMBER for batch loads, MERGE statements for idempotent upserts, insertId for streaming pipelines, and scheduled partition-level dedup jobs for cost efficiency.

If you want, I can show:

✅ Streaming + Dataflow dedup design

✅ Late-arriving data handling

✅ SCD2 with dedup

✅ Performance tuning examples

Just tell me 👍

Here are **frequently asked interview questions and topic areas** you should prepare for a **GCP Data Engineer role (4 years experience)** — especially relevant if you're interviewing at Capgemini (technical rounds focus on real project experience, SQL, GCP services, pipelines, and problem-solving) based on reported interview experiences and commonly asked GCP questions online:

📌 Capgemini GCP Data Engineer – Real Reported Questions

(From candidates who shared their actual interview experiences)

✅ Technical / Skill Questions

1. **Explain ETL basics** and how you implemented ETL in your project.

2. **Explain GCP services you've used** — BigQuery, Cloud Storage, Dataflow, Composer/Airflow, Pub/Sub.
3. **Write a BigQuery query to detect duplicates** in a table.
4. **BigQuery architecture and tables** (managed, external, partitioned, clustered).
5. Explain **Dataflow, Dataflow pipelines, and where you used them**.
6. Questions on **Python & SQL** related to data processing and pipeline logic.
7. **Your previous project details** — tools/tech used, transformations you implemented, window functions, broadcast joins, etc.

📌 Common GCP Data Engineer Questions

(Broader industry questions you should be ready for)

✔ GCP Core Concepts

8. What are the **key GCP services** relevant for data engineering (BigQuery, Dataflow, Pub/Sub, Dataproc, Cloud Storage)?
9. **How does BigQuery manage large datasets?** Explain columnar storage, slots and query performance.
10. **SQL optimization and query performance** in BigQuery.

🛠️ Architecture & Design

11. How would you **design a scalable data ingestion pipeline** on GCP? (Batch vs streaming)
12. How do you handle **schema evolution** in BigQuery datasets?
13. Describe **idempotent pipelines and late-arriving data** handling.
14. How would you structure a **multi-project GCP environment** for data teams?

📊 Pipeline & Data Quality

15. How do you **ensure data quality** and prevent duplicates or bad records?
16. What methods do you use for **monitoring and alerting** in production data pipelines?

17. How do you handle **data partitioning and clustering** for query performance?

🌐 GCP Services Deep Dive

18. Describe **Pub/Sub + Dataflow architecture** for real-time streaming.
19. Difference between **BigQuery and Cloud SQL/NoSQL** options.
20. Lifecycle and **data retention strategies** for GCS and BigQuery.

👨‍💼 Behavioral / Experience

21. Talk about a **problem you faced in a project** and how you solved it.
22. How do you **handle changes in requirements** under tight deadlines?
23. Explain **collaboration between technical and business teams**.

📌 Summary — Frequent Themes

✔ GCP core services (BigQuery, Dataflow, Pub/Sub)
✔ SQL + Python problems
✔ Real project scenarios
✔ Pipeline design & scalability
✔ Data quality & performance tuning
✔ Behavioral / communication skills

If you want, I can also **generate sample answers** or a **mock interview script** tailored to these questions. Just ask!

Based on the above questions, please provide short answers.

Below are **short, interview-ready answers** mapped to each question you received:

---

1) Explain ETL basics and your implementation

ETL extracts data from sources, transforms it (cleansing, dedup, enrichment), and loads into BigQuery. I used GCS → Dataflow → BigQuery for batch and Pub/Sub → Dataflow → BigQuery for streaming.

---

2) GCP services you've used

BigQuery (warehouse), GCS (storage), Dataflow (ETL), Pub/Sub (streaming), Composer (orchestration), Dataproc (Spark), Cloud SQL.

---

3) Detect duplicates in BigQuery

Use ROW_NUMBER with PARTITION BY business key and filter rn=1.

---

4) BigQuery table types

Managed tables store data in BigQuery, external tables reference external files, partitioned tables split by date, clustered tables organize by columns.

---

5) What is Dataflow

Managed Apache Beam service for batch and streaming ETL pipelines.

---

6) Python & SQL usage

Python for pipeline logic, transformations; SQL for aggregations, joins, window functions, and analytics.

---

7) Explain your project

Built ingestion pipelines, performed deduplication, transformations, and loaded curated datasets into BigQuery with monitoring.

---

8) Key GCP data services

BigQuery, Dataflow, Pub/Sub, Dataproc, GCS, Composer.

---

9) How BigQuery manages large datasets

Columnar storage, distributed execution, slots for parallelism.

---

10) SQL optimization in BigQuery

Use partitioning, clustering, filter early, avoid SELECT *, use approximate functions if possible.

---

11) Design scalable ingestion pipeline

GCS/PubSub → Dataflow → BigQuery, with autoscaling and retries.

---

12) Schema evolution handling

Use nullable columns, schema auto-update, versioned tables.

---

13) Idempotent pipelines & late data

Use MERGE on keys and event_time-based watermarking.

---

14) Multi-project structure

Separate projects for dev, test, prod with shared networking.

---

15) Ensure data quality

Validation rules, dedup logic, null checks, schema checks.

---

16) Monitoring & alerting

Cloud Monitoring, logs, pipeline metrics, failure alerts.

---

17) Partitioning & clustering

Partition by date, cluster by frequently filtered columns.

---

18) Pub/Sub + Dataflow architecture

Events → Pub/Sub → Dataflow streaming → BigQuery.

---

19) BigQuery vs Cloud SQL

BigQuery is analytical, Cloud SQL is transactional.

---

20) Retention strategies

GCS lifecycle rules, BigQuery partition expiration.

---

21) Problem you solved

Handled duplicates and late data using MERGE and window functions.

---

22) Handling changing requirements

Agile approach, quick POCs, modular pipelines.

---

23) Collaboration with stakeholders

Regular syncs, clarify requirements, demo outputs.

⭐ **One-line Interview Summary**

I build scalable batch and streaming pipelines on GCP using Dataflow, BigQuery, and Pub/Sub, focusing on data quality, performance, and reliability.

If you want, I can next provide **sample SQL snippets**, **mock interview Q&A**, or a **1-page cheat sheet** for quick revision.

sample SQL snippets, mock interview Q&A, or a 1-page cheat sheet

Great — here are **all three**, optimized for fast interview revision:

✅ **SAMPLE SQL SNIPPETS (BigQuery)**

Deduplicate Latest Record

```
SELECT *
FROM (
  SELECT *,
       ROW_NUMBER() OVER(PARTITION BY order_id ORDER BY updated_at
DESC) rn
```

```
  FROM orders
)
WHERE rn = 1;
```

Find Duplicate Keys

```
SELECT order_id, COUNT(*) cnt
FROM orders
GROUP BY order_id
HAVING cnt > 1;
```

Incremental Load (MERGE)

```
MERGE target t
USING staging s
ON t.id = s.id
WHEN MATCHED THEN UPDATE SET t.col = s.col
WHEN NOT MATCHED THEN INSERT ROW;
```

SCD Type 2

```
UPDATE dim
SET end_date = CURRENT_DATE, is_current = FALSE
WHERE id IN (SELECT id FROM staging);

INSERT INTO dim
SELECT *, CURRENT_DATE, NULL, TRUE
FROM staging;
```

Latest Record Per Group

```
SELECT *
FROM (
 SELECT *,
       RANK() OVER(PARTITION BY cust_id ORDER BY ts DESC) r
 FROM logs
)
```

```
WHERE r=1;
```

🎤 MOCK INTERVIEW Q&A

Q: How do you optimize BigQuery?
Use partitioning, clustering, pruning, and avoid SELECT *.

Q: Batch vs Streaming?
Batch = large periodic loads. Streaming = near real-time events.

Q: Why Dataflow?
Serverless, autoscaling, Apache Beam unified model.

Q: How handle schema changes?
Add nullable columns, versioning.

Q: How ensure exactly-once?
MERGE + idempotent keys.

Q: How handle failures?
Retries + dead-letter topics.

Q: Cost control?
Partition filters, storage lifecycle, slot reservations.

📄 ONE-PAGE CHEAT SHEET

Core Stack

- GCS → Dataflow → BigQuery
- Pub/Sub → Dataflow → BigQuery

BigQuery

- Partition by date
- Cluster by keys
- Use MERGE
- Window functions

Dataflow

- ParDo
- GroupByKey

- Windowing

Data Quality

- Dedup
- Null checks
- Schema validation

Performance

- Filter early
- Avoid cross joins
- Partition pruning

Security

- IAM roles
- Service accounts
- CMEK

⭐ 30-Second Elevator Pitch

I design scalable batch and streaming pipelines on GCP using Dataflow, Pub/Sub, and BigQuery with strong focus on data quality, deduplication, incremental loading, and cost optimization.

CSV files are typically received through SFTP, cloud storage buckets, application exports, APIs, or manual uploads. An automated ETL process detects new files, validates them, loads data into raw tables, archives processed files, and logs load status.

how many times in a day you are receiving the cav file

There is **no fixed number** — CSV file frequency depends on business needs and source system design.

In real projects, you'll usually see one of these patterns:

🔷 1) Once Per Day (Most Common)

- Nightly batch export
- Example: orders_20250206.csv
- Used for reporting, finance, historical analysis

👉 **Frequency:** 1 time/day

🔷 **2) Multiple Times Per Day**

- Every hour / every 4 hours
- Used when near-real-time insights are needed

👉 **Frequency:** 6–24 times/day

🔷 **3) Near Real-Time (Micro-Batches)**

- Every 5–15 minutes
- Small CSV files generated frequently

👉 **Frequency:** 96–288 times/day

🔷 **4) On-Demand / Event-Based**

- When partner triggers export
- Not on a fixed schedule

👉 **Frequency:** Variable

✅ **Typical Interview Answer**

We usually receive CSV files **once per day** for batch reporting, but in some integrations we receive them **hourly or every 15 minutes** depending on SLA. The pipeline is designed to handle both fixed-schedule and event-driven arrivals.

✅ **Best Practice**

Design pipeline to be **frequency-agnostic**:

- Detect new file
- Process automatically
- Track last processed file

Our workflow starts with detecting incoming CSV files, validating structure, loading into raw tables, cleansing and deduplicating in staging, applying SCD logic for dimensions, loading fact tables, running quality checks, and finally archiving files with logging and alerts.

volume of data

Good question 👍 — **data volume in CSV-based e-commerce pipelines** varies widely depending on company size, traffic, and frequency.

Here are **realistic ranges** you can confidently mention in interviews:

🔷 Small Scale

- 10k – 100k rows/day
- File size: 5–50 MB
- Example: small online store

🔷 Medium Scale

- 500k – 5 million rows/day
- File size: 200 MB – 2 GB
- Example: growing marketplace

🔷 Large Scale

- 10 million – 100 million rows/day
- File size: 2 GB – 20 GB
- Example: national e-commerce company

🔷 Very Large / Enterprise Scale

- 100 million – 1+ billion rows/day
- File size: 20 GB – 200+ GB
- Usually split into multiple files or moved to streaming

In my experience, CSV volumes range from a few hundred thousand to tens of millions of records per day. The pipeline is designed to scale using chunked loads, incremental processing, and partitioned tables so it can handle growth without redesign.

challenges loading data to bq

Here are the **most common real-world challenges when loading data into BigQuery** (especially from files, streaming, or ETL pipelines) — plus how teams typically solve them.

1️⃣ Schema Mismatch / Schema Drift

Problem

- New columns appear
- Data type changes (INT → STRING)
- Missing columns

Impact

- Load job fails
- Bad data lands

Solution

- Enable schema evolution (where appropriate)
- Maintain staging layer
- Validate schema before load

Interview line:

We handle schema drift using staging tables and schema validation before loading into final BigQuery tables.

2️⃣ Large File Size / High Volume

Problem

- Very large CSV/JSON files
- Slow loads / timeouts

Solution

- Split files
- Use parallel loads
- Prefer compressed files (GZIP)
- Partition tables

Interview line:

For large volumes we split files, use parallel loads, and load into partitioned tables.

3️⃣ Duplicate Records

Problem

- Same record arrives multiple times
- Retry jobs cause duplication

Solution

- Deduplicate in staging
- Use MERGE with primary key
- Idempotent loads

Interview line:

We use MERGE statements and primary-key based deduplication.

4️⃣ Bad / Corrupt Records

Problem

- Missing columns
- Broken rows
- Wrong delimiter

Solution

- Max bad records threshold
- Reject tables
- Data quality checks

Interview line:

Bad records are redirected to reject tables and investigated separately.

5️⃣ Data Type Conversion Errors

Problem

- String instead of date
- Invalid timestamps

Solution

- Load as STRING in raw
- Cast in staging with SAFE_CAST

Interview line:

We ingest raw as string and safely cast in transformation layer.

6️⃣ Cost Overruns

Problem

- Full table reloads
- Large scans

Solution

- Incremental loads
- Partitioning
- Clustering

Interview line:

We control cost using incremental loads and partitioned tables.

7️⃣ Late Arriving Data

Problem

- Old dates arrive today
- Missing historical updates

Solution

- Reprocess recent partitions
- Watermark logic

Interview line:

We allow reprocessing of last N days to capture late data.

8️⃣ Streaming vs Batch Consistency

Problem

- Streaming data arrives before batch
- Duplicate events

Solution

- Unified dedup logic
- Event_id based merging

Interview line:

We unify streaming and batch data using event_id-based deduplication.

9️⃣ Load Job Failures

Problem

- Network issues
- Quota exceeded

Solution

- Retry logic
- Alerts
- Monitoring

Interview line:

Automated retries and alerts handle transient failures.

1️⃣0️⃣ Table Locking / Concurrency

Problem

- Multiple jobs writing same table

Solution

- Write to staging
- Atomic MERGE to final table

Interview line:

We isolate writes using staging tables and atomic MERGE operations.

⭐ One Strong Interview Summary

Key challenges include schema drift, large volumes, duplicates, bad records, type mismatches, and cost control. We solve these using staging layers, incremental loads, MERGE-based upserts, partitioned tables, data quality checks, and monitoring in **Google Cloud BigQuery**.